EXP NO: 3	CLASSIFICATION WITH DECISION TREES

AIM:

To implement a Decision Tree classifier on the Iris dataset and evaluate its performance using the accuracy score and confusion matrix.

ALGORITHM:

1. Import necessary libraries.
2. Load a classification dataset (Iris) using load_iris().
3. Split the dataset into training and test sets (70% / 30%).
4. Preprocess data if needed:
   - Not required here, since Decision Trees do not need feature scaling and Iris has no missing or categorical values.
5. Train a DecisionTreeClassifier (criterion='gini') from sklearn.tree.
6. Predict on the test data.
7. Evaluate using:
   - Confusion Matrix
   - Accuracy Score
8. Visualize the confusion matrix and the Decision Tree using plot_tree().

CODE:

```
# 1. Install necessary libraries (if not already installed)
# pip install numpy pandas matplotlib seaborn scikit-learn

# Step 1: Import Libraries
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load Dataset
iris = load_iris()
X = iris.data
y = iris.target
print(iris.data[:5])    # first 5 rows of features
print(iris.target)

# Step 3: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Step 4: Train the Decision Tree Classifier
dt_model = DecisionTreeClassifier(criterion='gini', random_state=0)
dt_model.fit(X_train, y_train)

# Step 5: Predict
y_pred = dt_model.predict(X_test)

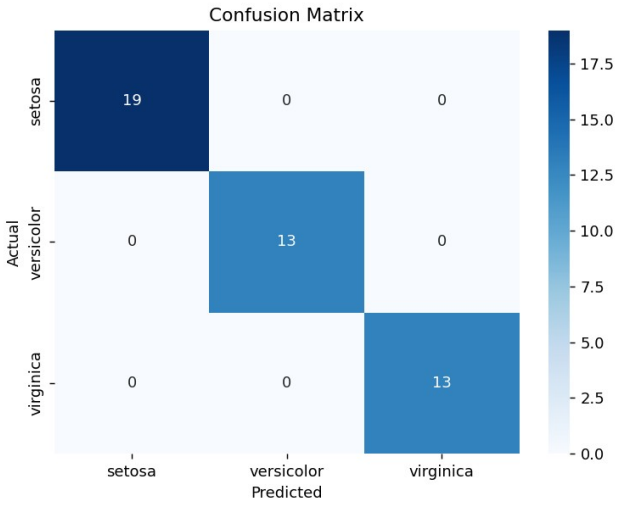
# Step 6: Evaluate the Model
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("Accuracy Score:", acc)

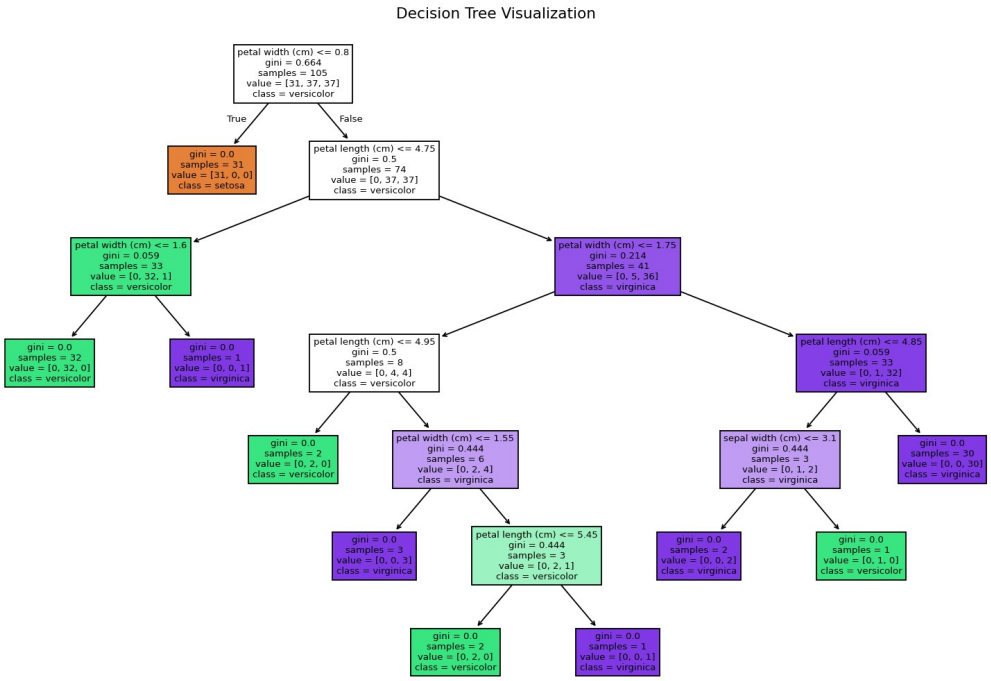
# Step 7: Visualize Confusion Matrix
sns.heatmap(cm, annot=True, cmap="Blues",
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# Step 8: Visualize the Decision Tree
plt.figure(figsize=(12, 8))
plot_tree(dt_model, filled=True,
          feature_names=iris.feature_names,
          class_names=iris.target_names)
plt.title("Decision Tree Visualization")
plt.show()
```

OUTPUT:

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2]
Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
Accuracy Score: 1.0
```





RESULT:

The Decision Tree classifier was successfully implemented on the Iris dataset. The model achieved an accuracy of 100% on the test set, with all 45 test samples classified correctly (19 Setosa, 13 Versicolor and 13 Virginica), as shown by the confusion matrix. The visualized tree (depth 6, 10 leaves) splits mainly on petal width and petal length, making the decision rules easy to interpret. Thus, Decision Trees are effective and interpretable for classification problems.